دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه سیزدهم: جدول آدرس‌دهی مستقیم و جدول درهم‌سازی

دکتر شهرام گلزاری

پاییز ۱۴۰۵

اهداف جلسه

در پایان این جلسه انتظار داریم که:

- برای اعمال فرهنگ لغت داده‌ساختاری متناسب با نیازهای خود طراحی کنیم.
- حد پایین جستجو با مقایسه را اثبات کنیم.
- با دانستن اندازه‌ی جهان کلیدها و تعداد کلیدهای ذخیره‌شده، تشخیص دهیم که جدول آدرس‌دهی مستقیم عملی است یا جدول درهم‌سازی لازم است.
- روی جدول آدرس‌دهی مستقیم، رویه‌ای برای یک پرسش تازه (مانند یافتن بیشینه یا پذیرفتن کلید تکراری) طراحی و زمان بدترین حالت آن را تعیین کنیم.
- با داشتن تابع درهم‌ساز و دنباله‌ای از کلیدها، خانه‌ی هر کلید و همه‌ی برخوردها را پیدا کنیم.
- وضعیت جدول درهم‌سازی با زنجیره‌سازی را پس از دنباله‌ای از درج، جستجو و حذف رسم و تعداد مقایسه‌های هر جستجو را تعیین کنیم.

چرا درهم‌سازی

اعمال فرهنگ لغت

«در بسیاری از کاربردها نیاز است که بر روی مجموعه‌ی پویایی از داده‌ها اعمال فرهنگ داده‌ای، یعنی جست‌وجو، درج و حذف انجام شود. مجموعه‌ای را پویا می‌گوییم که تعداد عناصر آن در طول زمان تغییر کنند. مثلاً، چنان‌چه در فصل ۴ هم گفتیم، یک کامپایلر نیاز دارد کلمات کلیدی خود را در یک جدول نمادها سازمان‌دهی کند تا بتواند در ترجمه‌ی یک برنامه، بررسی کند که آیا تک‌تکِ کلمات استفاده شده در آن برنامه در جدول نمادها هست یا خیر.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۶۱

هزینه‌ی اعمال فرهنگ لغت در داده‌ساختارهای قبلی

در n عنصر، با ساختارهایی که تاکنون دیده‌ایم:

- جستجو:
- درج:
- حذف:

حد پایین جستجو با مقایسه

- در مدل مقایسه، الگوریتم عناصر را فقط با مقایسه از هم تمیز می‌دهد و کران پایین زمان اجرا برابر است با تعداد مقایسه‌های انجام شده.
- درخت تصمیم هر الگوریتم جستجو با مقایسه دست‌کم $n+1$ برگ دارد، پس ارتفاع آن $\Omega(\lg n)$ است.
- برای جستجوی سریع‌تر از $\lg n$ باید از چیزی جز مقایسه استفاده کنیم. بهره بردن از دسترسی تصادفی $O(1)$ در مدل رم.

MIT 6.006, Spring 2020، یادداشت‌های جلسه‌ی ۴ (ترجمه ماشینی) – *Link*

ایده‌ی اصلی

«جدول درهم‌سازی مفهوم (...) آرایه‌ی معمولی را تعمیم می‌دهد. آدرس‌دهی مستقیم در یک آرایه‌ی معمولی از زمان دسترسی $O(1)$ به هر عنصر آرایه بهره می‌برد.»

CLRS, 4th ed.، صفحه ۲۷۲ (ترجمه ماشینی)

- خود کلید را اندیس آرایه می‌گیریم: جدول آدرس‌دهی مستقیم.
- اندیس را از روی کلید محاسبه می‌کنیم: جدول درهم‌سازی.

جدول آدرس‌دهی مستقیم

آدرس‌دهی مستقیم

«آدرس‌دهی مستقیم روشی ساده است که وقتی جهان U کلیدها به اندازه‌ی کافی کوچک باشد به‌خوبی کار می‌کند. فرض کنید یک برنامه کاربردی به مجموعه‌ای پویا نیاز دارد که هر عنصر آن کلیدی یکتا دارد که از جهان $U = \{0, 1, \ldots, m-1\}$ انتخاب شده است و m خیلی بزرگ نیست.»

CLRS, 4th ed.، صفحه ۲۷۳ (ترجمه ماشینی)

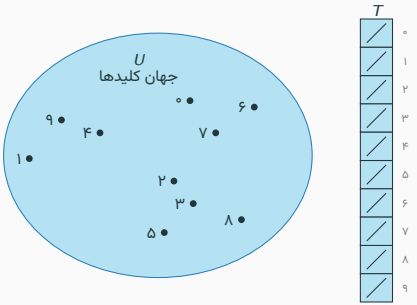
- جهان کلیدها (universe): همه‌ی کلیدهای ممکن.
- K: کلیدهایی که واقعاً در مجموعه هستند.

جدول آدرس‌دهی مستقیم

«برای نمایش مجموعه‌ی پویا می‌توان از یک آرایه یا جدول آدرس‌دهی مستقیم (direct-address table) به نام $T[0 : m - 1]$ استفاده کرد که هر مکان یا خانه‌ی (slot) آن متناظر با یک کلید در جهان U است. (...) خانه‌ی k به عنصری از مجموعه با کلید k اشاره می‌کند. اگر مجموعه هیچ عنصری با کلید k نداشته باشد، آنگاه $T[k] = \text{nil}$.»

CLRS, 4th ed.، صفحه ۲۷۳ (ترجمه ماشینی)

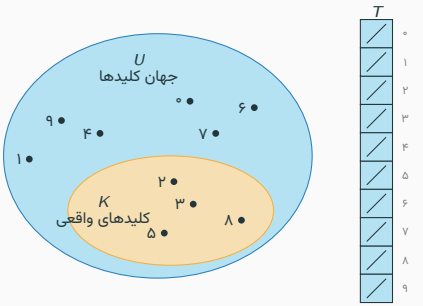
نمایش با آرایه



«هر کلید در جهان $U = \{۰, ۱, \ldots, ۹\}$ متناظر با یک اندیس در جدول است.»

CLRS, 4th ed.، صفحه ۲۷۳ (بازترسیم شکل ۱.۱۱، ترجمه ماشینی شرح شکل)

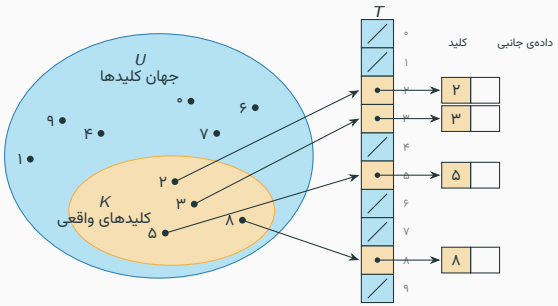
نمایش با آرایه



«مجموعه‌ی $K = \{۲, ۳, ۵, ۸\}$ از کلیدهای واقعی، خانه‌هایی از جدول را تعیین می‌کند که اشاره‌گر به عناصر دارند.»

CLRS, 4th ed.، صفحه ۲۷۳ (بازترسیم شکل ۱۱.۱، ترجمه ماشینی شرح شکل)

نمایش با آرایه



«بقیه‌ی خانه‌ها، به رنگ آبی، مقدار nil دارند.»

CLRS, 4th ed.، صفحه ۲۷۳ (بازترسیم شکل ۱۱.۱، ترجمه ماشینی شرح شکل)

اعمال جدول آدرس‌دهی مستقیم

```
DIRECT-ADDRESS-SEARCH(T, k)
1  return T[k]

DIRECT-ADDRESS-INSERT(T, x)
1  T[x.key] = x

DIRECT-ADDRESS-DELETE(T, x)
1  T[x.key] = NIL
```

CLRS, 4th ed.، صفحه ۲۷۴

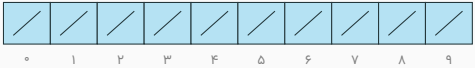
«پیاده‌سازی اعمال فرهنگ لغت DIRECT-ADDRESS-SEARCH، DIRECT-ADDRESS-INSERT و DIRECT-ADDRESS-DELETE (...) بدیهی است. هر کدام فقط زمان $O(1)$ می‌گیرد.»

CLRS, 4th ed.، صفحه ۲۷۳ (ترجمه ماشینی)

این $O(1)$ در بدترین حالت است و به هیچ مقایسه‌ای میان کلیدها نیاز ندارد.

ردیابی یک مثال

جدول با $m = ۱۰$ و دنباله‌ی اعمال زیر:



جدول خالی: همه‌ی خانه‌ها nil هستند.

ردیابی یک مثال

جدول با $m = ۱۰$ و دنباله‌ی اعمال زیر:



درج عنصری با کلید ۵: $T[5] = x$، یک دسترسی به آرایه.

ردیابی یک مثال

جدول با $m = ۱۰$ و دنباله‌ی اعمال زیر:



درج عنصری با کلید ۲: $T[2] = x$.

ردیابی یک مثال

جدول با $m = ۱۰$ و دنباله‌ی اعمال زیر:



جستجوی کلید ۳: $T[3]$ برابر nil است، پس کلید ۳ در مجموعه نیست. تنها یک دسترسی لازم بود.

ردیابی یک مثال

جدول با $m = ۱۰$ و دنباله‌ی اعمال زیر:



حذف عنصر با کلید ۵: $T[5] = \text{NIL}$.

جدول درهم‌سازی

عیب آدرس‌دهی مستقیم

«عیب آدرس‌دهی مستقیم آشکار است: اگر جهان U بزرگ یا نامتناهی باشد، ذخیره‌ی جدول T به اندازه‌ی $|U|$ با توجه به حافظه‌ی موجود در یک رایانه‌ی معمولی ممکن است غیرعملی یا حتی ناممکن باشد. افزون بر این، مجموعه‌ی K از کلیدهایی که واقعاً ذخیره می‌شوند ممکن است نسبت به U چنان کوچک باشد که بیشتر فضای اختصاص‌یافته به T هدر برود.»

CLRS, 4th ed.، صفحه ۲۷۵ (ترجمه ماشینی)

مثال

نگهداری اطلاعات دانشجویان یک دانشگاه با کلید کد ملی

صرفه‌جویی در فضا

وقتی $|K|$ بسیار کوچک‌تر از $|U|$ است، جدول درهم‌سازی فضای بسیار کمتری می‌گیرد. «(...) به‌طور مشخص، فضای لازم به $\Theta(|K|)$ کاهش می‌یابد و در عین حال این مزیت حفظ می‌شود که جستجوی یک عنصر در جدول درهم‌سازی همچنان فقط زمان $O(1)$ لازم دارد. نکته این است که این کران برای زمان حالت میانگین است، در حالی که برای آدرس‌دهی مستقیم در بدترین حالت برقرار است.»

CLRS, 4th ed.، صفحه ۲۷۵ (ترجمه ماشینی)

	آدرس‌دهی مستقیم	جدول درهم‌سازی
فضا	$\Theta(\|U\|)$	$\Theta(\|K\|)$
جستجو	$O(1)$ در بدترین حالت	$O(1)$ در حالت میانگین

تابع درهم‌ساز

«در آدرس‌دهی مستقیم، عنصری با کلید k در خانه‌ی k ذخیره می‌شود، اما در درهم‌سازی از یک تابع درهم‌ساز h (hash function) استفاده می‌کنیم تا شماره‌ی خانه را از روی کلید k محاسبه کنیم و عنصر در خانه‌ی $h(k)$ قرار گیرد. تابع درهم‌ساز h جهان U کلیدها را به خانه‌های یک جدول درهم‌سازی (hash table) $T[0 : m-1]$ نگاشت می‌کند:

$$h : U \to \{0, 1, \ldots, m-1\},$$

که در آن اندازه‌ی m جدول درهم‌سازی معمولاً بسیار کمتر از $|U|$ است.»

CLRS, 4th ed.، صفحه ۲۷۵ (ترجمه ماشینی)

مقدار هش

«می‌گوییم عنصری با کلید k به خانه‌ی $h(k)$ هش می‌شود (hashes) و نیز می‌گوییم $h(k)$ مقدار هش (hash value) کلید k است. (...) نمونه‌ای از یک تابع درهم‌ساز ساده، اما نه چندان خوب، $h(k) = k \bmod m$ است.»

CLRS, 4th ed.، صفحه ۲۷۵ (ترجمه ماشینی)

مثال

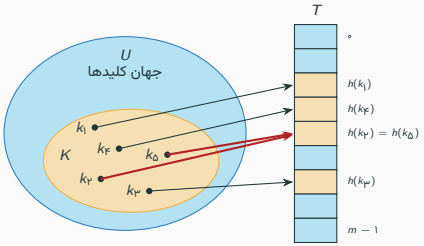
با $m = ۱۰$ داریم $h(۲۳) = ۳$ و $h(۷۱) = ۱$. پس جدول فقط ۱۰ خانه دارد، هر قدر هم جهان کلیدها بزرگ باشد.

برخورد

«یک مشکل وجود دارد، و آن اینکه دو کلید ممکن است به یک خانه هش شوند. این وضعیت را برخورد (collision) می‌نامیم. خوشبختانه روش‌های مؤثری برای حل تعارض ناشی از برخوردها وجود دارد.»

CLRS, 4th ed.، صفحه ۲۷۵ (ترجمه ماشینی)

در شکل، کلیدهای k_2 و k_5 به یک خانه نگاشته شده‌اند و برخورد دارند.



CLRS, 4th ed.، صفحه ۲۷۶ (بازترسیم شکل ۲.۱۱)

اجتناب‌ناپذیری برخورد

«با این حال، چون $|U| > m$، دست‌کم دو کلید باید مقدار هش یکسانی داشته باشند و اجتناب کامل از برخوردها ناممکن است. بنابراین، هرچند یک تابع درهم‌ساز خوب‌طراحی‌شده و «تصادفی‌نما» می‌تواند تعداد برخوردها را کم کند، همچنان به روشی برای حل برخوردهایی که رخ می‌دهند نیاز داریم.»

CLRS, 4th ed.، صفحه‌های ۲۷۵ و ۲۷۶ (ترجمه ماشینی)

پاراگراف بالا کدام یک از اصول ریاضی را بیان می‌کند؟

یافتن برخوردها

با $m = ۱۰$ و $h(k) = k \bmod ۱۰$، کلیدهای $۲۳, ۴۵, ۱۳, ۷۱, ۵۰, ۳۳$ را هش می‌کنیم:

k	۲۳	۴۵	۱۳	۷۱	۵۰	۳۳
$h(k)$	۳	۵	۳	۱	۰	۳

خانه اثر دارد و این یکی از دلایلی است که $k \bmod m$ تابع چندان خوبی نیست.

یافتن برخوردها

با $m = ۱۰$ و $h(k) = k \bmod ۱۰$، کلیدهای $۲۳, ۴۵, ۱۳, ۷۱, ۵۰, ۳۳$ را هش می‌کنیم:

k	۲۳	۴۵	۱۳	۷۱	۵۰	۳۳
$h(k)$	۳	۵	۳	۱	۰	۳

کلیدهای ۲۳، ۱۳ و ۳۳ همگی به خانه‌ی ۳ درهم می‌شوند.

خانه اثر دارد و این یکی از دلایلی است که $k \bmod m$ تابع چندان خوبی نیست.

یافتن برخوردها

با $m = ۱۰$ و $h(k) = k \bmod ۱۰$، کلیدهای $۲۳, ۴۵, ۱۳, ۷۱, ۵۰, ۳۳$ را هش می‌کنیم:

k	۲۳	۴۵	۱۳	۷۱	۵۰	۳۳
$h(k)$	۳	۵	۳	۱	۰	۳

کلیدهای ۲۳، ۱۳ و ۳۳ همگی به خانه‌ی ۳ درهم می‌شوند.

اکنون کلیدهای $۱۰, ۲۰, ۳۰, ۴۰$ را در نظر بگیرید: همه به خانه‌ی ۰ می‌روند. با $m = ۱۰$ فقط رقم آخر کلید در خانه اثر دارد و این یکی از دلایلی است که $k \bmod m$ تابع چندان خوبی نیست.

درهم‌سازی یکنواخت مستقل

«یک تابع درهم‌ساز «ایده‌آل» h برای هر ورودی ممکن k در دامنه‌ی U، خروجی $h(k)$ دارد که عنصری است که به‌طور تصادفی، مستقل و یکنواخت از بازه‌ی $\{0, 1, \ldots, m-1\}$ انتخاب شده است. وقتی مقداری برای $h(k)$ به‌طور تصادفی انتخاب شد، هر فراخوانی بعدی h با همان ورودی k همان خروجی $h(k)$ را می‌دهد.»

CLRS, 4th ed.، صفحه ۲۷۶ (ترجمه ماشینی)

- چنین تابعی تابع درهم‌ساز یکنواخت مستقل (independent uniform hash function) یا پیشگوی تصادفی (random oracle) نامیده می‌شود.
- این یک فرض نظری برای تحلیل است و در عمل فقط تقریب زده می‌شود.

حل برخورد با زنجیره‌سازی

ایده‌ی زنجیره‌سازی

در زنجیره‌سازی (chaining)، «(...) هر خانه‌ی غیرخالی به یک لیست پیوندی اشاره می‌کند و همه‌ی عناصری که به یک خانه هش می‌شوند به لیست پیوندی آن خانه می‌روند. خانه‌ی j اشاره‌گری به سر لیست همه‌ی عناصر ذخیره‌شده با مقدار هش j دارد. اگر چنین عنصری نباشد، خانه‌ی j مقدار nil دارد.»

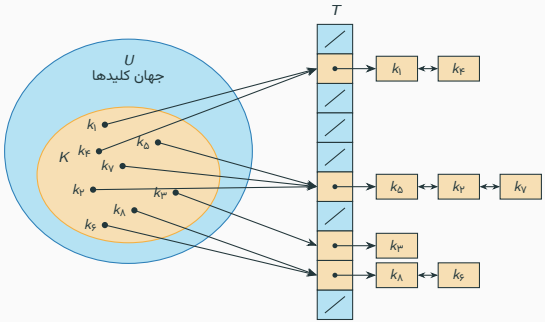
CLRS, 4th ed.، صفحه ۲۷۷ (ترجمه ماشینی)

نگاه کلی

«در سطح بالا، می‌توان درهم‌سازی با زنجیره‌سازی را شکلی غیربازگشتی از تقسیم و حل دانست: مجموعه‌ی ورودی n عنصری به‌طور تصادفی به m زیرمجموعه، هر کدام با اندازه‌ی تقریبی n/m، تقسیم می‌شود.»

CLRS, 4th ed.، صفحه ۲۷۷ (ترجمه ماشینی)

شکل زنجیره‌سازی



«حل برخورد با زنجیره‌سازی. هر خانه‌ی غیرخالی $T[j]$ جدول درهم‌سازی به یک لیست پیوندی از همه‌ی کلیدهایی اشاره می‌کند که مقدار هش آن‌ها j است. برای مثال $h(k_1) = h(k_4)$ و $h(k_2) = h(k_5) = h(k_7)$.»

CLRS, 4th ed.، صفحه ۲۷۷ (بازترسیم شکل ۳.۱۱، ترجمه ماشینی شرح شکل)

اعمال با زنجیره‌سازی

```
CHAINED-HASH-INSERT(T, x)
1  LIST-PREPEND(T[h(x.key)], x)

CHAINED-HASH-SEARCH(T, k)
1  return LIST-SEARCH(T[h(k)], k)

CHAINED-HASH-DELETE(T, x)
1  LIST-DELETE(T[h(x.key)], x)
```

CLRS, 4th ed.، صفحه ۲۷۸

- هر خانه یک لیست پیوندی است و هر عمل فقط روی لیست خانه‌ی $h(\cdot)$ انجام می‌شود.
- رویه‌های LIST-PREPEND، LIST-SEARCH و LIST-DELETE همان رویه‌های لیست پیوندی هستند که پیش‌تر دیدیم.

زمان اجرای اعمال

«زمان اجرای درج در بدترین حالت $O(1)$ است. رویه‌ی درج تا حدی به این دلیل سریع است که فرض می‌کند عنصر x که درج می‌شود از قبل در جدول نیست. (...) برای جستجو، زمان اجرا در بدترین حالت متناسب با طول لیست است. (...) اگر لیست‌ها دوطرفه باشند، حذف زمان $O(1)$ می‌گیرد.»

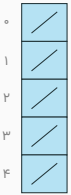
CLRS, 4th ed.، صفحه ۲۷۷ (ترجمه ماشینی)

سوال

چرا لیست دو طرفه؟

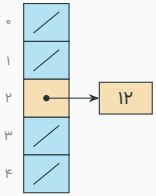
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

جدول خالی با ۵ خانه.



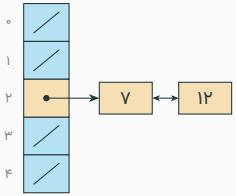
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۱۲: $h(۱۲) = ۲$.



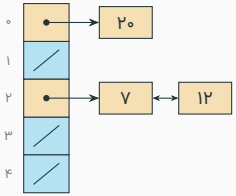
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۷: $h(۷) = ۲$، برخورد با ۱۲. عنصر ۷ به ابتدای لیست خانه‌ی ۲ افزوده می‌شود.



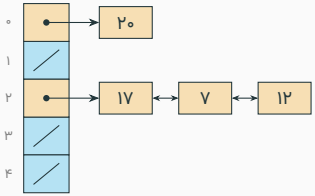
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۲۰: $h(۲۰) = ۰$.



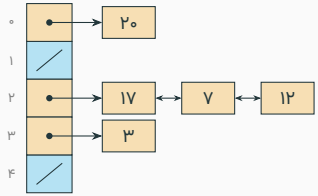
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۱۷: $h(۱۷) = ۲$. لیست خانه‌ی ۲ اکنون سه عنصر دارد.



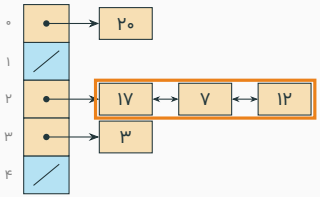
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۳: $h(۳) = ۳$.

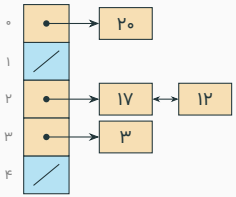


$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

جستجوی ۲۲: $h(۲۲) = ۲$. کلیدهای ۱۷، ۷ و ۱۲ مقایسه می‌شوند و پس از ۳ مقایسه، ناموفق بودن جستجو معلوم می‌شود.



$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.



حذف عنصر با کلید ۷ با داشتن اشاره‌گر به آن: پیوندهای ۱۷ و ۱۲ مستقیم به هم وصل می‌شوند، در زمان $O(۱)$.

منابع

منابع

- محمد قدسی، داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- Link E. Demaine, J. Ku, J. Solomon, *6.006 Introduction to Algorithms*, Spring 2020, MIT OpenCourseWare.